

---

# Credit Risk Intelligence

## Project Documentation

Group DomainRange

UIU Data Analytics Laboratory

Musfique Ahmed · Tasfiya Binte Karim

---

Home Credit Default Risk · 307,511 loans · XGBoost AUC 0.7578

## 1. Project Goal

Build an end-to-end loan-default forecasting system on the Home Credit Default Risk dataset (`application_train.csv` — 307,511 loans × 122 columns). The output is a deployed scoring function plus an interactive dashboard that recommends Approve / Manual Review / Reject for any new applicant.

The dataset is heavily imbalanced (~8% default rate), mixes numeric and categorical features, and has substantial missingness — making it a realistic credit-risk modeling exercise.

---

## 2. Phase 1 — Data Cleaning & Preparation

Source files: `src/data/clean.py`, `reports/data_quality_report.md`, `reports/cleaning_summary.md`

### What we did

- Replaced the `DAYS_EMPLOYED == 365243` sentinel with `NaN` (55,374 rows — a known "missing" code in the raw data).
  - Dropped 17 columns with >60% missingness (mostly normalized housing-info fields: `COMMONAREA_*`, `LIVINGAPARTMENTS_*`, `YEARS_BUILT_*`, etc.).
  - Imputed the remaining 51 columns with `NaNs` — numeric with median, categorical with the literal `'MISSING'`.
  - Encoded:
    - 13 low-cardinality categoricals → one-hot.
    - 2 high-cardinality categoricals (`OCCUPATION_TYPE`, `ORGANIZATION_TYPE`) → frequency encoding.
  - Fitted `StandardScaler` on the train slice only to prevent leakage.
  - Stratified split (preserves the 92/8 target balance):
    - Train: 215,257 rows
    - Val: 46,127 rows
    - Test: 46,127 rows
  - Output: `data/processed/train_clean.parquet` → 307,511 × 144 columns.
- 

## 3. Phase 2 — Exploratory Data Analysis

Source files: `notebooks/01_eda.ipynb` (10 sections, 23 figures) → `reports/eda_findings.md`

### 3.1 What we explored

1. Univariate distributions — income, credit, age, employment length.
2. Bivariate plots — each feature against default rate.
3. `EXT_SOURCE` deep-dive — three external credit-bureau scores.
4. Employment / income vs. risk.
5. Occupation / education / contract type segments.

6. Correlation heatmap across numeric features.
7. Three formal hypothesis tests (below).
8. Key insights narrative.
9. Chart export → reports/figures/01..12\_\*.png.

## 3.2 Three hypothesis tests

| ID | Hypothesis                                       | Result                                                                                                                          |
|----|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| H1 | Lower EXT_SOURCE → higher default                | SUPPORTED — $r \approx -0.16$ to $-0.18$ , $p \approx 0$                                                                        |
| H2 | Higher credit-to-income ratio → higher default   | ❑ NOT supported as stated — inverted-U shape; Simpson's paradox from EXT_SOURCE confound                                        |
| H3 | Occupation/education segments have elevated risk | SUPPORTED — Low-skill Laborers 17.2% vs Accountants 4.8% (3.5× spread); education shows clean monotonic gradient (10.9% → 1.8%) |

## 3.3 Key findings (with numbers)

- EXT\_SOURCE\_3 is the single most actionable external signal. It should be required on every credit application.
- Credit-to-income (CTI) is non-monotonic. Default rate peaks at 9.2% in decile 6 and falls to 7.1% at the top. When you condition on EXT\_SOURCE\_3, the sign of the CTI coefficient flips — a textbook Simpson's paradox driven by EXT\_SOURCE confounding the naive CTI effect.
- Segment-level risk is real but small in effect size. Chi-squared p-values are astronomical (much less than  $1e-200$ ), but Cramér's V is only  $\sim 0.03$ – $0.08$ . Segments are useful as adjuncts, not standalone decision rules.
- Income dominates among employment / income axes. Income verification should be prioritized in the underwriting workflow.
- Class imbalance: 92/8. Even a univariate AUC of 0.65 (the EXT\_SOURCE mean alone) leaves substantial residual risk — multi-feature models are required.

## 3.4 Caveats

- All findings are observational — no causal claims.
- Phase 4 quantifies predictive lift with proper cross-validation.

# 4. Phase 3 — Feature Engineering & Selection

Source files: src/features/engineer.py, src/features/select.py, notebooks/02\_feature\_selection.ipynb, reports/feature\_importance.md

## 4.1 Engineered features (7)

- AGE\_YEARS — DAYS\_BIRTH / -365
- EMPLOYED\_YEARS — DAYS\_EMPLOYED / -365
- EXT\_SOURCE\_MEAN — mean of 3 EXT\_SOURCE scores (partial-NaN-safe)
- CREDIT\_INCOME\_RATIO — AMT\_CREDIT / AMT\_INCOME\_TOTAL
- ANNUITY\_INCOME\_RATIO — AMT\_ANNUITY / AMT\_INCOME\_TOTAL

- CREDIT\_GOODS\_RATIO — AMT\_CREDIT / AMT\_GOODS\_PRICE
- Plus additional domain-derived ratios from the builder.

## 4.2 Top-30 feature selection

Combined three rankings into a single mean-rank / combined-score:

- XGBoost rank
- Random Forest rank
- SHAP rank

Top-5 selected features:

| Rank | Feature         | Mean rank |
|------|-----------------|-----------|
| 1    | EXT_SOURCE_MEAN | 1.0       |
| 2    | EXT_SOURCE_1    | 7.3       |
| 3    | EMPLOYED_YEARS  | 9.3       |
| 4    | EXT_SOURCE_3    | 10.7      |
| 5    | CODE_GENDER_M   | 11.0      |

## 4.3 Baseline vs. reduced comparison

5-fold CV on:

- All features LR: AUC =  $0.7445 \pm 0.0026$
- Top-30 LR: AUC =  $0.7397 \pm 0.0023$

Top-30 is within ~0.005 AUC of all-features — we kept top-30 for the dashboard's tractable form input.

# 5. Phase 4 — Model Selection & Comparison

Source files: `src/models/train.py`, `notebooks/03_model_building.ipynb`,  
`reports/phase4_model_comparison.md`

## 5.1 Why these 4 models?

The master prompt required "4+ ML/DL models." We picked a deliberately diverse set covering different model families, feature-handling assumptions, and interpretability:

| Model               | Family            | Why we chose it                                                                                                                           |
|---------------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Logistic Regression | Linear baseline   | Interpretable, fast, gives a coefficient baseline. Necessary sanity check.                                                                |
| Random Forest       | Bagged trees      | Strong out-of-the-box baseline; handles NaN natively; gives feature importance.                                                           |
| XGBoost             | Gradient boosting | Industry standard for tabular data; known to win Kaggle-style structured problems. Chosen over LightGBM for API parity with scikit-learn. |
| MLP (PyTorch)       | Deep learning     | Provides the DL family. PyTorch chosen because TensorFlow has no Python 3.14 wheels.                                                      |

## 5.2 Imbalance strategy decision

| Family              |        | SMOTE  |
|---------------------|--------|--------|
| Logistic Regression | 0.7398 | 0.7394 |
| XGBoost             | 0.7539 | 0.7203 |

SMOTE was tried and rejected. It dropped XGBoost val AUC from 0.7539 → 0.7203 — a clear regression. We stuck with class weighting.

## 5.3 Final 4-model comparison (validation)

| Model               | Imbalance  | Config                                                  | Val AUC | Precision | Recall | F1    |
|---------------------|------------|---------------------------------------------------------|---------|-----------|--------|-------|
| XGBoost (winner)    | balanced   | n_est=300, max_depth=4, lr=0.05, scale_pos_weight=11.39 | 0.7539  | 0.164     | 0.675  | 0.264 |
| Random Forest       | balanced   | n_est=300, max_depth=None                               | 0.7469  | 0.237     | 0.394  | 0.296 |
| MLP (PyTorch)       | pos_weight | hidden=[128, 64], dropout=0.3, epochs=12                | 0.7448  | 0.153     | 0.688  | 0.250 |
| Logistic Regression | balanced   | C=10.0                                                  | 0.7398  | 0.156     | 0.660  | 0.252 |

## 5.4 Outcomes

- Winner: XGBoost — Val AUC 0.7539, Test AUC 0.7578 at threshold 0.500.
- XGBoost had the best balance of AUC and recall (0.675), the highest among gradient-boosting-style models on this data.
- Saved as models/best\_model.pkl (joblib pickle, 511 KB, gitignored).

## 5.5 Recommendation policy

Encoded in src/models/score.py:

| Probability of default | Recommendation                            |
|------------------------|-------------------------------------------|
| $p < 0.2$              | Approve (clearly low risk)                |
| $0.2 \leq p < 0.5$     | Manual Review (gray zone — human decides) |
| $p \geq 0.5$           | Reject (clearly high risk)                |

## 5.6 Three synthetic applicant demos

| Profile   | Probability | Recommendation |
|-----------|-------------|----------------|
| Low-risk  | 0.0073      | Approve        |
| Mid-risk  | 0.3865      | Manual Review  |
| High-risk | 0.9548      | Reject         |

## 6. Phase 5 — Interactive Dashboard

Source files: `dashboard/app.py`, `dashboard/README.md`

A 4-tab Streamlit app wired live to the trained XGBoost model. It serves at `http://localhost:8501`.

### 6.1 Tabs

| Tab                         | What it does                                                                                                                   |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Portfolio Overview          | KPI cards (215,257 apps, 8.07% default rate, 17,377 defaults) + class balance + 4-model AUC bar chart + 3-way policy explainer |
| Applicant Risk Scorer       | 14-widget form → live XGBoost score → hero card (Approve / Manual Review / Reject) + segmented gauge + sensitivity expander    |
| Feature Importance Explorer | Top-30 features with category filter (EXT_SOURCE / Time / One-hot / Ratio / Amount / Area)                                     |
| Segment Analysis            | Default rate by income decile / employment bucket / region / occupation with 95% CI error bars                                 |

### 6.2 Design

- Dark navy palette #0A1428 with mint #00D9B5, blue #3B82F6, red #F87171 accents.
- Inter for body type, JetBrains Mono for numerics.
- All theming lives in `dashboard/_theme.py` — single source of truth.

### 6.3 Stack note

Streamlit pinned to `>=1.38, <1.50` because Streamlit 1.50+ migrated to an ASGI stack with a WebSocket-after-handshake regression on Python 3.14 that leaves the browser stuck on skeleton placeholders.

## 7. Phase 6 — Documentation, Tests, Reproducibility

### 7.1 Tests (23 passing)

| File                                 | Tests | What it covers                                              |
|--------------------------------------|-------|-------------------------------------------------------------|
| <code>tests/test_clean.py</code>     | 3     | Phase 1 cleaning + scaler-fit-on-train-only leakage check   |
| <code>tests/test_engineer.py</code>  | 6     | Phase 3 feature engineering edge cases (partial-NaN safety) |
| <code>tests/test_score.py</code>     | 8     | Phase 4 scoring function schema and policy thresholds       |
| <code>tests/test_dashboard.py</code> | 6     | Phase 5 in-process Streamlit AppTest smoke test             |

### 7.2 Reproduction pipeline

```
.venv/Scripts/python notebooks/_build_eda_notebook.py      # Phase 2
.venv/Scripts/python notebooks/_build_phase3_notebook.py   # Phase 3
.venv/Scripts/python notebooks/_build_phase4_notebook.py   # Phase 4 → writes models/best_model.pkl
.venv/Scripts/python -m streamlit run dashboard/app.py      # Phase 5 → localhost:8501
```

---

## 8. Final Outcomes — At a Glance

### 8.1 Quantitative outcomes

- Best model: XGBoost — Val AUC 0.7539, Test AUC 0.7578.
- Top-5 features: EXT\_SOURCE\_MEAN, EXT\_SOURCE\_1, EMPLOYED\_YEARS, EXT\_SOURCE\_3, CODE\_GENDER\_M.
- Imbalance strategy winner: `class_weight='balanced'` (not SMOTE).
- Test/val gap: +0.0039 — model is not overfitting.

### 8.2 Qualitative outcomes

- Deployed scoring function — `score_application(input: dict) -> dict` returning probability + recommendation.
- Interactive dashboard with 4 tabs covering portfolio, scoring, feature importance, and segment analysis.
- Transparent documentation — `README.md`, `dashboard/README.md`, `models/README.md`, `docs/PHASE_CHECKLIST.md`, `docs/PROJECT_DOCUMENTATION.md`.
- Honest flagging — no chat-style "AI agent" was built; that gap is documented in the phase checklist rather than silently inflated.

### 8.3 Business takeaways

1. EXT\_SOURCE\_3 should be a required field on every credit application — single most actionable signal.
  2. Avoid simple CTI thresholds — high-CTI applicants are also high-income and high-EXT-SOURCE; a calibrated model captures the nuance.
  3. Segment-level underwriting is defensible but should sit alongside individual features rather than serve as the sole decision rule.
  4. Income verification should be prioritized in underwriting workflows.
- 

## 9. Repo Layout

```
.
├── dashboard/           # 4-tab Streamlit app (app.py + helpers + README.md)
├── data/processed/      # parquet artifacts (gitignored) + .gitkeep
├── docs/                # PHASE_CHECKLIST.md, PROJECT_DOCUMENTATION.md
├── models/              # best_model.pkl (gitignored) + README.md
├── notebooks/           # 3 .ipynb + canonical builder .py scripts
├── reports/             # 5 .md findings + figures/ (23 PNGs)
├── src/
│   ├── data/            # load + clean + profile
│   ├── features/        # engineer + select
│   └── models/          # train + tune + evaluate + score + mlp.py
├── tests/               # 23 pytest tests across 4 files
├── puku_master_prompt.md # project specification
├── requirements.txt
└── README.md
```

## 10. Authors & Acknowledgements

- Course: UIU Data Analytics Laboratory
- Group: DomainRange
- Members: Musfique Ahmed, Tasfiya Binte Karim
- Dataset: Home Credit Default Risk (Kaggle) — application data only
- Project brief: `puku_master_prompt.md` in this repo

The full plan + per-phase deliverables are tracked in `docs/PHASE_CHECKLIST.md`.